GUÍA PASO A PASO + CAPTURAS

UNITY 6 + REALIDAD VIRTUAL EN NASCOR

Del proyecto 3D al XR Origin funcional: simulación con teclado y ratón, seguimiento con Meta Quest Link, controles, manos animadas, locomoción continua, rayos y teletransporte.

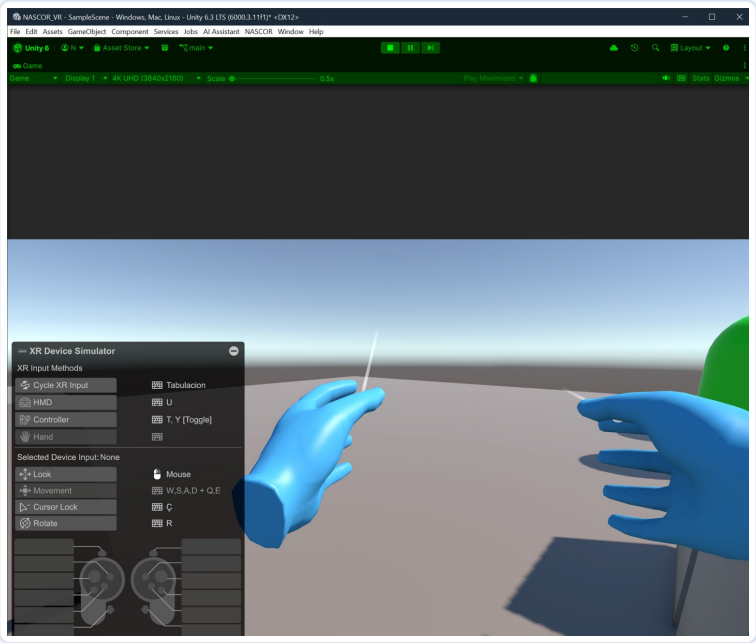


Imagen real de la prueba en Unity: escenario NASCOR y manos animadas con XR Device Simulator.

ESTADO DOCUMENTADO

El **simulador funcionó** con movimiento continuo, cambio manos/mandos y teletransporte. Con **Meta Quest Link se visualizó la escena**. El punto de aparición al iniciar con visor físico quedó **pendiente**; no se da por resuelto.

Basado en capturas y ajustes de tu sesión real, 29 de octubre de 2026. Los nombres exactos de ciertas opciones dependen de la versión instalada de XR Interaction Toolkit.

01 · Qué construimos

Mapa de componentes y resultados, antes de entrar en los detalles de los ajustes.

Dos formas de ejecutar el MISMO proyecto

SIMULADOR (SIN GAFAS)	META QUEST LINK (GAFAS REALES)
XR Device Simulator genera entradas del HMD y controladores a través de Input System.	OpenXR utiliza el visor y mandos físicos a través del runtime Meta Horizon Link.
No requiere que arranque el subsistema XR de PC; se prueban cámara, manos y controles en Play.	Requiere OpenXR activo, perfil Oculus Touch y conexión Quest Link; se inicia la escena actual con Play.
Confirmado: movimiento WASD, rayos, teletransporte y cambio visual de manos.	Confirmado: renderizado del escenario. Quedó por validar por separado el seguimiento visible de manos y el punto de inicio.

Mapa de la mecánica de teletransporte

Elemento	Responsabilidad
XR Origin (XR Rig)	Jugador VR; cámara rastreada y objetos de manos/controladores.
Controller Input Action Manager	Gestiona las acciones de movimiento, giro y Teleport Mode.
XR Ray Interactor + Line Visual	Raycast nativo, arco y colores de línea.
Teleportation Area + Collider	Marca superficies que aceptan teletransporte.
Teleportation Provider + Locomotion Mediator	Ejecuta el cambio de posición del jugador.

NO PARTIMOS DE CERO

El prefab **XR Origin (XR Rig)** de Starter Assets ya trae casi toda la infraestructura. No fue necesario programar un Physics.Raycast manual ni un controlador de desplazamiento.

02 · Preparación del proyecto

Lo que se revisó antes de que NASCOR pudiera ejecutar Play.

Antes de tocar XR: compilar limpio

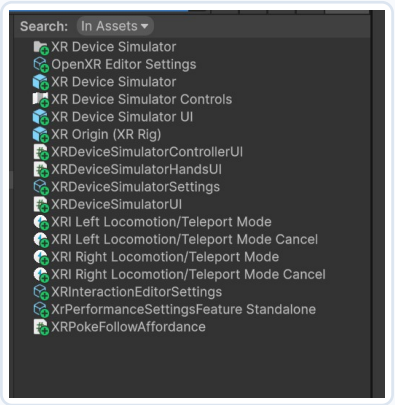
- Abrimos el NASCOR existente con **Unity 6.3 LTS (6000.3.11f1)**; no creamos una escena nueva.
- Los errores **CS0101/CS0111** procedían de clases de la comparativa repetidas en distintos scripts; la comparativa no forma parte del objetivo VR.
- Para una práctica VR limpia se apartan los scripts conflictivos de **Assets**, conservando respaldo y archivos **.meta** cuando corresponda.
- Al final del proceso se veían mensajes en consola ajenos al sistema de teletransporte, incluidos avisos **Mono.Cecil** y “Falta InputComparativaMovimiento en el padre”. Conviene limpiarlos posteriormente sin deshacer la configuración XR.

Paquetes en Window > Package Manager

Paquete	Papel en NASCOR
Input System	Acciones y bindings para simulación y mandos físicos.
XR Plug-in Management	Gestiona proveedores XR y el arranque automático.
OpenXR Plugin	Comunicación con el runtime XR de Windows.
XR Interaction Toolkit	Interactores, locomoción, teletransporte y utilidades.

Samples que importamos

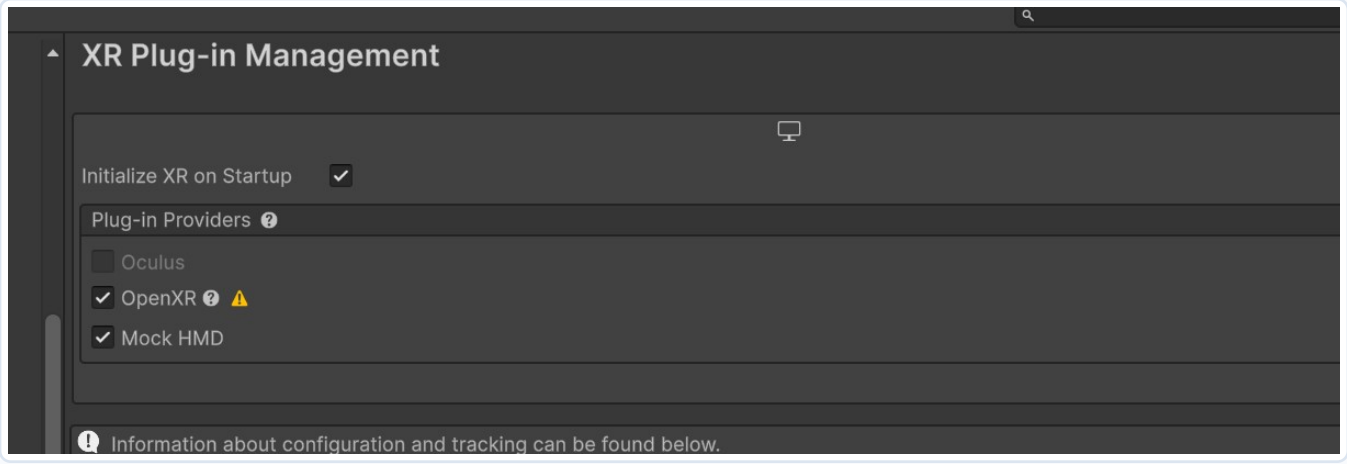
- **Starter Assets:** prefab XR Origin, XRI Default Input Actions y componentes de movimiento/teletransporte ya preparados.
- **XR Device Simulator:** prefab clásico con controles para teclado/ratón. No hay que confundirlo con el nuevo XR Interaction Simulator.



Búsqueda en Assets: XR Device Simulator y XR Origin (XR Rig) aparecían como prefabs utilizables.

03 · Ajustes XR en Windows

Dos modos desde el mismo editor: simulación y conexión real Quest Link.



Project Settings > XR Plug-in Management, pestaña Windows: OpenXR y Mock HMD aparecían activados inicialmente.

Configuración deseada por modo

Opción	Simulador clásico	Quest Link
Initialize XR on Startup	No	Sí
OpenXR	Puede quedar asignado, sin inicializar	Sí, proveedor activo
Mock HMD	No hace falta	No
XR Device Simulator (escena)	Sí, solo uno	No

IMPORTANTE

Mock HMD no equivale a XR Device Simulator. Mock HMD simula un visor; el XR Device Simulator crea entradas virtuales de visor y mandos. Para nuestro recorrido habitual, Mock HMD no es necesario.

Conmutador NASCOR > VR (script Editor)

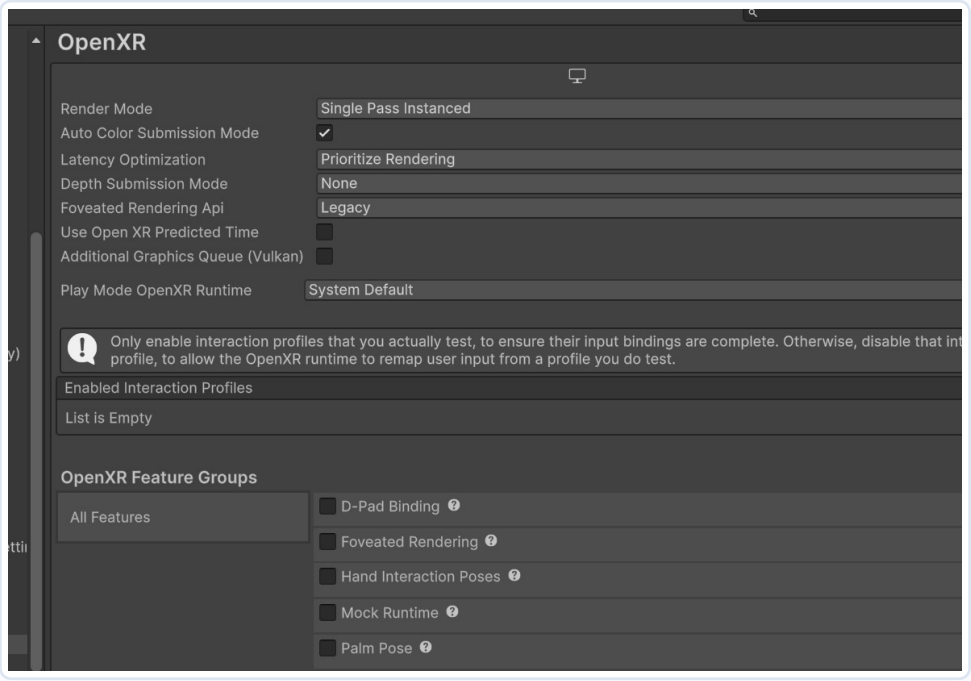
Se incorporó **Assets/Editor/NASCOR_VR_ModosEditor.cs** para cambiar entre simulador y Quest Link fuera de Play. La versión actualizada automatiza la casilla de arranque XR y activa/desactiva el prefab del simulador existente.

```
NASCOR > VR > 1 - Simulador (teclado y raton)
NASCOR > VR > 2 - Meta Quest Link (gafas reales)
NASCOR > VR > 3 - Mock HMD (experimental)
```

El script del Editor **no puede por sí mismo establecer la sesión del visor en Quest Link ni cambiar el runtime de Windows**. Las opciones deben conmutarse fuera de Play y guardarse con Ctrl+S.

04 · OpenXR y los mandos Quest

Este fue el ajuste que faltaba cuando vimos el escenario en las gafas pero no las manos.



OpenXR inicialmente mostraba “Enabled Interaction Profiles: List is Empty”.

Ajuste necesario para los mandos físicos

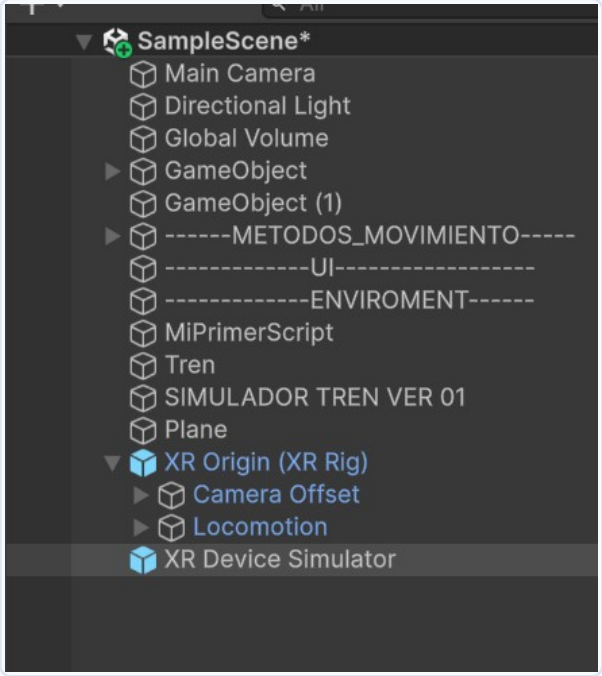
- Edit > Project Settings > XR Plug-in Management > **OpenXR**.
- Seleccionar **Windows, Mac, Linux**, no Android para probar desde Play por Quest Link.
- En **Enabled Interaction Profiles**, pulsar **+** y añadir **Oculus Touch Controller Profile**. Este perfil se añadió durante las pruebas.
- Revisar **Project Validation** y corregir los requisitos obligatorios de OpenXR; si la versión incluye un grupo de funciones Meta Quest aplicable, revisar su disponibilidad.
- El XR Origin debe disponer de **Input Action Manager** con el asset **XRI Default Input Actions** habilitado.

RESULTADO QUE SÍ VERIFICAMOS

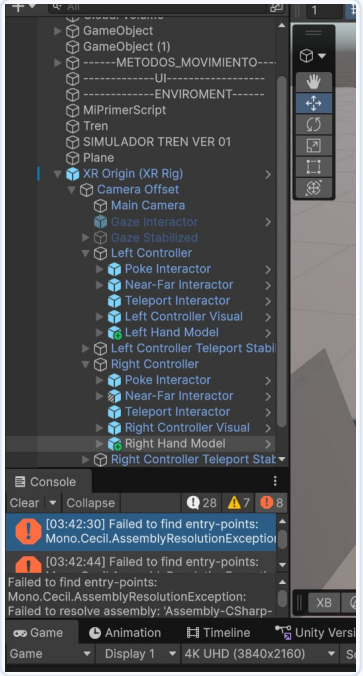
Tras conectar Quest Link, Unity mostró correctamente la imagen estereoscópica y los objetos del escenario. La confirmación de los **controladores visibles en las Quest reales** se dejó pendiente en ese momento; no debemos confundirlo con las manos ya probadas en simulador.

05 · XR Origin y jerarquía

Cómo añadimos el prefab correcto sin rehacer la escena original.



Primer montaje: XR Origin y XR Device Simulator añadidos a SampleScene.



Estructura desplegada: Camera Offset, interactores, controladores y Locomotion.

Qué permanece en la escena

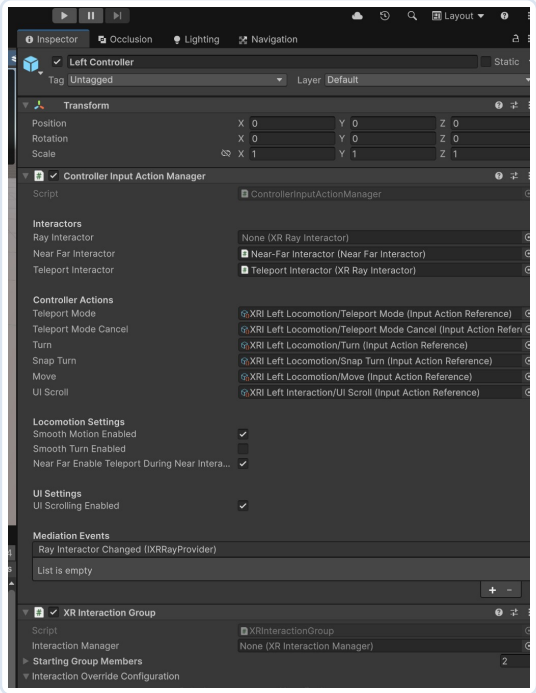
- El **XR Origin (XR Rig)** se arrastró desde Starter Assets. Evitamos crear otro XR Origin adicional.
- Se desactivó la **Main Camera antigua** porque el XR Origin tiene su propia cámara dentro de Camera Offset.
- El **XR Device Simulator** va como objeto independiente en la raíz de la escena.
- El **Plane** y los elementos antiguos del escenario permanecen en NASCOR.

Componentes útiles del prefab

Camera Offset: referencia de seguimiento. **Tracked Pose Driver:** aplica posición/rotación del HMD y los mandos. **XR Input Modality Manager:** gestiona modos de mano/controlador. **XR Interaction Manager:** interacciones. **XR Interaction Group:** arbitra los interactores disponibles.

06 · Controladores e Input Actions

Los mandos izquierdo y derecho comparten estructura, pero pueden usar distintas locomociones.



Inspector real del controlador izquierdo: Controller Input Action Manager, XR Interaction Group, Haptic Impulse Player y Tracked Pose Driver.

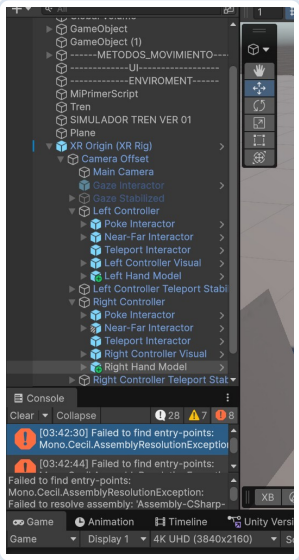
Acciones clave que vimos

Acción / componente	Función
XRI Left/Right · Position, Rotation, Tracking State	Seguimiento XR de cada controlador físico o simulado.
XRI Right Locomotion/Teleport Mode	Activa el modo de teletransporte del mando derecho.
Move y Turn	Movimiento continuo y giro.
Teleport Interactor	Rayo especializado que selecciona zonas válidas.
Poke / Near-Far Interactor	Interacciones de proximidad o a distancia, distintas del teletransporte.

En el **Right Controller**, desactivamos **Smooth Motion Enabled** para reservar su locomoción al teletransporte. El izquierdo conservó movimiento continuo. El controlador muestra **Teleport Interactor** correctamente asignado; **Ray Interactor = None** no demuestra por sí solo un fallo si hay un interactor específico de teletransporte.

07 · Manos riggeadas y mandos visuales

Aprovechamos los modelos animados del curso sin tocar el seguimiento XR.



Debajo de cada controlador se añadieron modelos de mano propios, además del visual del mando original.

Colocación de manos

- Añadir **Left Hand Model** como hijo de **Left Controller** y **Right Hand Model** como hijo de **Right Controller**.
- Copiar las posiciones/rotaciones locales recomendadas en los prefabs del curso; no aplicar Transform genéricos si el modelo ya trae un offset correcto.
- Conservar **Animator**, mallas y rig; desactivar solo objetos **Left/Right Controller Visual** si deseamos mostrar exclusivamente manos.
- Las manos de malla son **representaciones visuales**; no equivalen automáticamente al seguimiento de dedos sin mandos mediante XR Hands.

Alternador visual: V en PC / A en Quest

Añadimos **AlternarManosMandosXR.cs** sobre **XR Origin**, con cuatro referencias: dos manos y dos visuales de mandos. Solo cambia su visibilidad mediante **SetActive**; no desactiva el sistema de tracking ni los interactores.

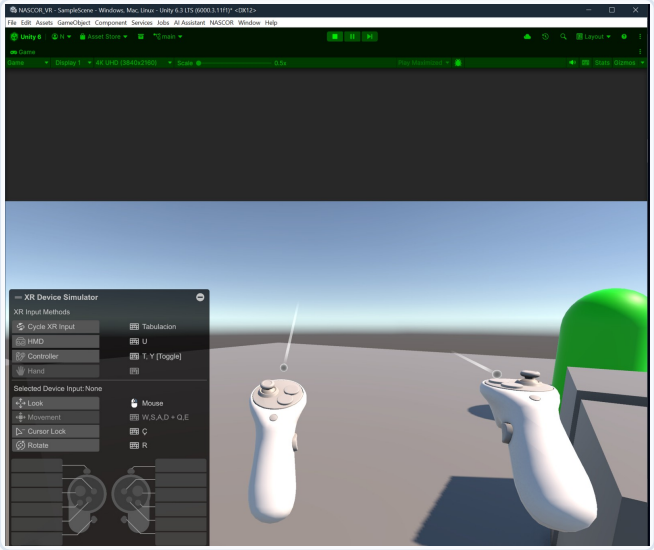
```
manoIzquierda.SetActive(mostrandoManos);
manoDerecha.SetActive(mostrandoManos);
mandoIzquierdo.SetActive(!mostrandoManos);
mandoDerecho.SetActive(!mostrandoManos);
```

PROBADO

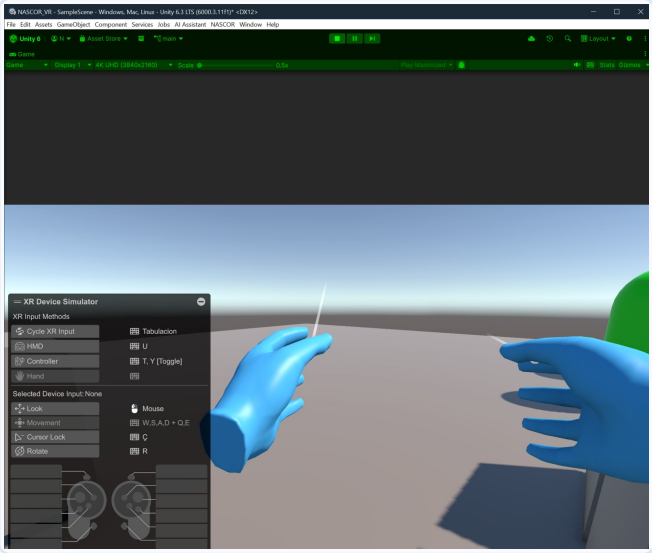
En el simulador se vio la alternancia entre modelos de controladores blancos y manos azules. La animación individual de dedos se configura aparte, según el Animator del paquete.

08 · Simulador de dispositivos

El primer objetivo que quedó operativo: navegar sin ponernos las gafas.



XR Device Simulator mostrando el escenario con visuales de mandos.



El mismo escenario mostrando las manos del curso, tras alternar con V.

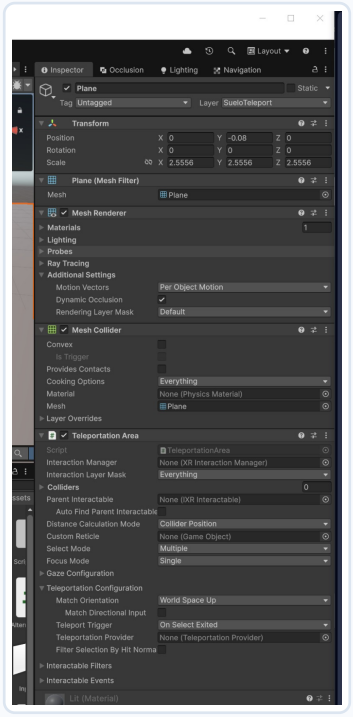
Prueba de funcionamiento

- Con **NASCOR > VR > Simulador**, desactivar la inicialización XR y dejar activo un único XR Device Simulator en la escena.
- Entrar en **Play** y enfocar la ventana Game. La interfaz del simulador muestra el mapa efectivo de teclas.
- Se verificó **WASD** para caminar continuamente, cámara y controladores simulados, y tecla **V** para alternar visuales.
- Importante: el simulador clásico puede reasignar teclas al dispositivo seleccionado; conviene guiarse por su panel actual, no por combinaciones genéricas.

> **CONFUSIÓN QUE RESOLVIMOS**
>
> La tecla **W** por sí sola movía al personaje hacia delante; no era la orden de teletransporte. Evitamos volver a asociar “W = teletransporte” porque en esta escena está ocupada por el movimiento continuo.

09 · Plane, Collider y Layers

La zona de suelo necesita dos filtros distintos: física y XR.



Plane: Layer física SueloTeleport, Mesh Collider y Teleportation Area; el estado inicial mostrado tenía Interaction Layer Mask = Everything.

Dos capas, dos funciones

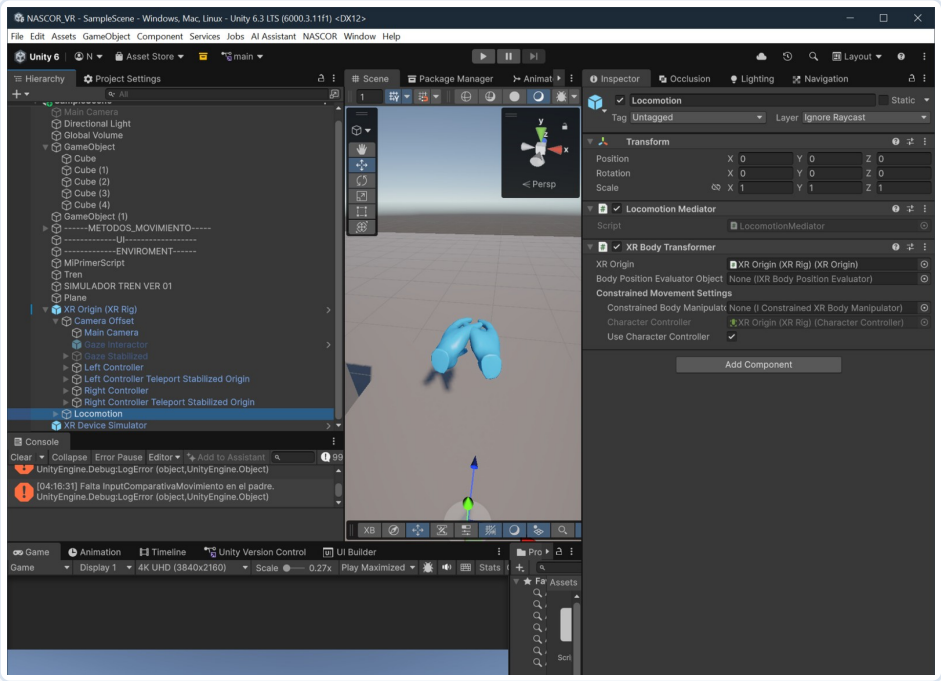
Capa	Función	Ajuste recomendado
Physics Layer	Controla qué Colliders impacta el raycast del XR Ray Interactor.	Plane: SueloTeleport; interactor: Raycast Mask = SueloTeleport.
XR Interaction Layer	Controla si el interactor puede seleccionar el Teleportation Area.	Area e interactor comparten Teleport.

El **Mesh Collider** del Plane está activado. La lista **Colliders = 0** en Teleportation Area no implica fallo si el componente detecta el Collider asociado. El valor inicial **Everything** en XR Interaction Layer también incluye Teleport; luego recomendamos usar Teleport específicamente para reducir cruces de interacciones.

> **LIMITACIÓN DEL FILTRO**
>
> Si Raycast Mask solo contiene SueloTeleport, el rayo **ignora obstáculos de otras capas**. Es lo que buscábamos en la práctica; en escenarios de zombis hay que decidir si debe bloquearse el teletransporte a través de paredes.

10 · Locomotion y Teleportation Provider

No se programa el desplazamiento: el Toolkit lo resuelve con componentes.



En el Inspector real de Locomotion estaban Locomotion Mediator y XR Body Transformer; no se veía un Teleportation Provider en ese objeto.

Configuración que revisamos

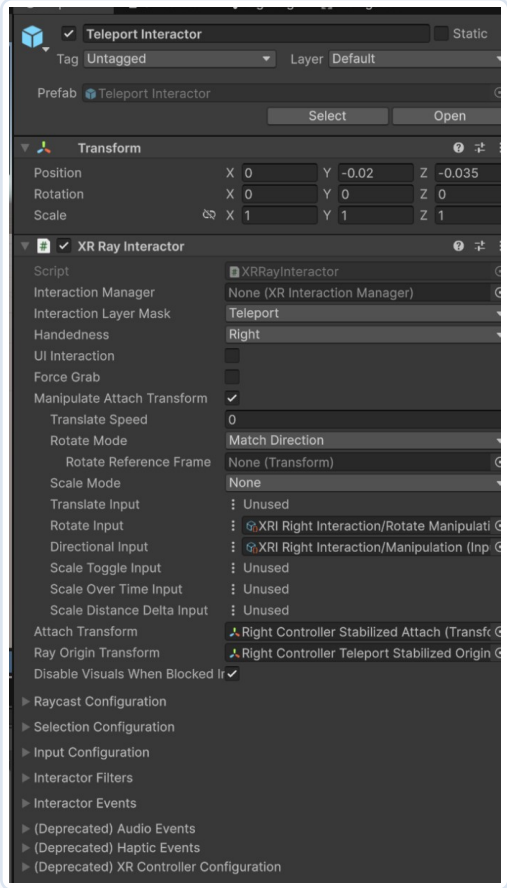
- En **XR Origin > Locomotion**, verificar que existan **Locomotion Mediator** y **XR Body Transformer**.
- Si no hay un **Teleportation Provider** en la jerarquía, añadirlo al objeto Locomotion (o a un hijo dedicado) y asignar el Mediator correspondiente.
- En **Plane > Teleportation Area > Teleportation Provider** se puede asignar explícitamente el proveedor disponible; la búsqueda automática puede estar admitida, pero la referencia explícita ayuda al diagnóstico.
- Mantener **Teleport Trigger = On Select Exited** si queremos confirmar al soltar la acción, como en los Starter Assets.

Diferencia entre los tres componentes de locomoción

Componente	Responsabilidad
Locomotion Mediator	Coordina a los proveedores de locomoción.
XR Body Transformer	Aplica cambios al cuerpo XR Origin.
Teleportation Provider	Solicita y ejecuta cambios de posición/rotación al teletransportar.

11 · Raycast nativo y arco de teletransporte

La detección y el dibujo del rayo ya existen en Starter Assets.



Right Controller > Teleport Interactor: Interaction Layer Mask = Teleport, Handedness = Right y origen estabilizado configurado.

Inspector: Raycast Configuration

Propiedad	Valor de la práctica
XR Ray Interactor	Activado.
Interaction Layer Mask	Teleport.
Raycast Mask	SueloTeleport (capa física).
Line Type	Projectile Curve, para arco en vez de línea recta.
XR Interactor Line Visual	Presente y activo; alimenta al Line Renderer.

El componente **XR Ray Interactor** lanza la consulta física y comprueba el interactable. **XR Interactor Line Visual** utiliza esos resultados para dibujar una línea válida o inválida. No fue necesario escribir **Physics.Raycast()**, **Instantiate()** ni mover el XR Origin a mano para la locomoción.

12 · Colores y confirmación

Dos cosas diferentes: activar el rayo y validar el destino.

Comportamiento esperado

Estado	Visual	Acción
Teleport Mode inactivo	El arco puede permanecer oculto	El jugador continúa usando locomoción normal.
Rayo toca Teleportation Area aceptada	Gradiente válido (azul en nuestra práctica)	Al confirmar/soltar: teletransporte.
Rayo sin destino válido	Gradiente inválido (rojo)	No se ejecuta el teletransporte.

¿Dónde se cambian los colores?

En **Right Controller > Teleport Interactor > XR Interactor Line Visual**, revisar los gradientes para línea válida e inválida y su habilitación. En las capturas iniciales el Line Renderer se veía, pero los gradientes quedaban fuera del recorte; su valor final debe confirmarse en tu Inspector.

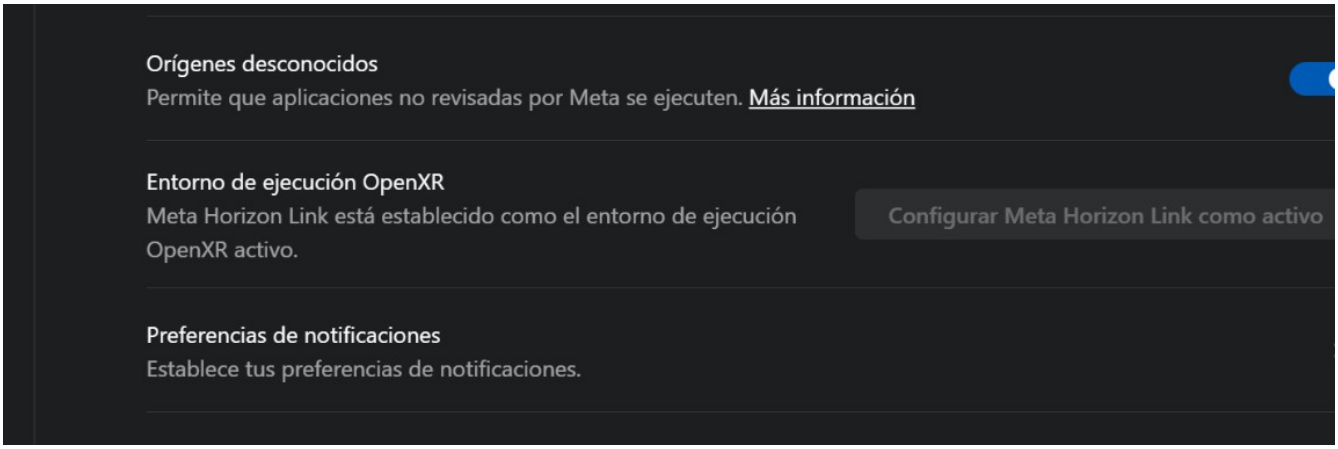
Cómo lo probamos con XR Device Simulator

- La acción usada por el controlador derecho era **XRI Right Locomotion/Teleport Mode**.
- W seguía reservada a caminar. Finalmente el modo de teletransporte se probó simulando el joystick del mando derecho con la combinación indicada por el panel del simulador: **seleccionar mando izquierdo con T y utilizar I** sobre el joystick contrario, con el mapeo activo en esa escena.
- Cuando el usuario confirmó “funciona perfecto tanto el continuo como el teleport”, dimos por **validada la locomoción en simulador**.

> **DEPENDE DEL BINDING**
>
> No codificar “I” como tecla universal para todos los proyectos: es el control observado en este simulador clásico con sus acciones. En gafas reales, manda el binding **Teleport Mode** del controlador Oculus Touch.

13 · Conectar Meta Quest Link

Cómo pasamos de Play simulado a Play desde las gafas reales.



Meta Horizon Link en Windows: el propio programa indicaba que ya era el runtime OpenXR activo.

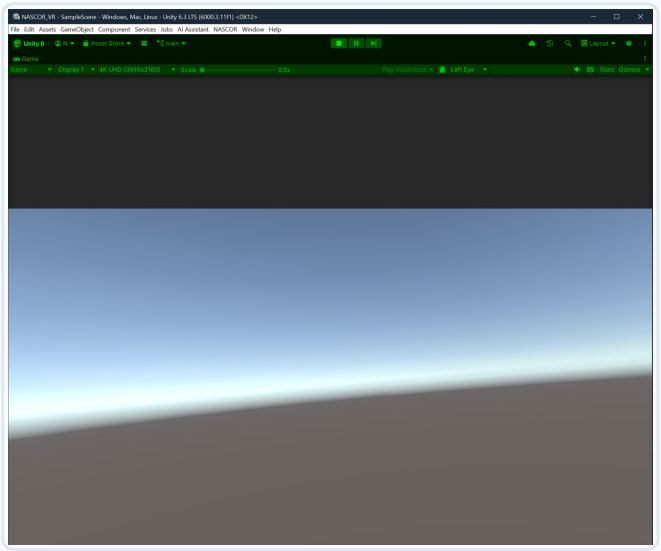
Pasos reproducibles

- Abrir **Meta Horizon Link** en Windows e iniciar sesión con la cuenta Meta vinculada. No hace falta cambiar Opera como navegador predeterminado.
- Comprobar en la aplicación que **Meta Horizon Link es el runtime OpenXR activo**; si el botón para activarlo está deshabilitado porque ya es activo, no hay nada que modificar.
- Fuera de Play, ejecutar **NASCOR > VR > Meta Quest Link**. El selector actualizado activa OpenXR y **Initialize XR on Startup** y desactiva XR Device Simulator.
- Confirmar **Oculus Touch Controller Profile** en Windows/OpenXR (lo añadimos más adelante al descubrir que estaba vacío).
- Conectar las Quest y entrar en **Quest Link** desde el visor; posteriormente pulsar **Play** en Unity. No es necesario cargar Unity desde la biblioteca de las Quest ni instalar APK para esta prueba.

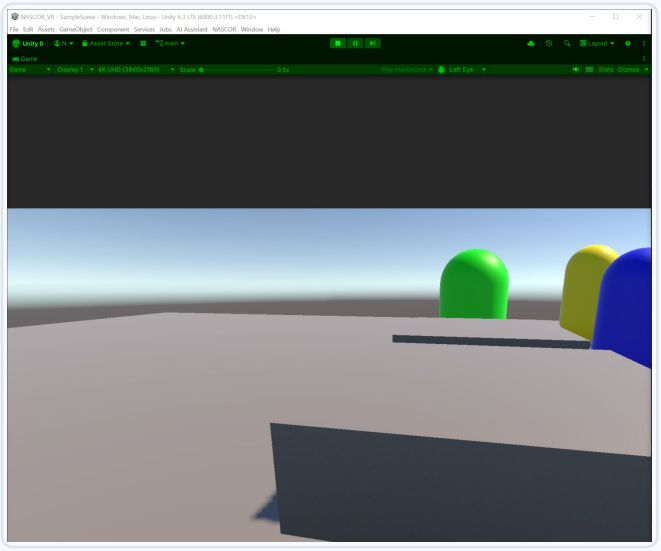
> **AVISO REAL DE LA APLICACIÓN**
>
> Se observó un aviso rojo de Meta Horizon Link sobre un problema conocido del controlador NVIDIA. Eso es ajeno a los scripts de Unity; si hay bloqueos, revisar controlador gráfico y notas de Meta antes de cambiar la configuración del proyecto.

14 · Primera prueba en las Quest

Lo que observamos realmente al ejecutar SampleScene con Quest Link.



Inicio: el visor mostraba el horizonte y parecía una escena vacía.



Al cambiar la orientación/posición del visor, apareció el escenario con las cápsulas y la geometría de NASCOR.

Diagnóstico que corrigió la confusión

La ventana Game mostraba **Left Eye**. Esto indica que Unity estaba visualizando la salida del ojo izquierdo; **la escena ya era SampleScene**. No faltaba configurarla como escena de inicio ni compilar una APK.

La razón de la vista aparentemente vacía era la **posición y/o orientación del seguimiento del visor** al arrancar, no una escena ausente. Con XR el Transform de la Main Camera hija recibe datos en tiempo real del visor y no necesariamente coincide con la cámara simulada.

¿Qué quedó pendiente?

- **Punto de aparición:** el inicio con las Quest sigue quedando desplazado o mal orientado. Se creó PuntoInicioVR e InicioVR.cs y se probó, pero el usuario aclaró después que aún instanciaba mal.
- **Manos reales:** se añadió Oculus Touch Controller Profile; no consta una confirmación final de que ambos mandos aparezcan y sigan correctamente sus poses en el visor tras el cambio.

15 · PuntoInicioVR (pendiente)

Cómo se intentó fijar el punto de inicio y qué falta comprobar.

Lo que hicimos

- Configuramos **XR Origin > Tracking Origin Mode = Floor**. Esto afecta la referencia de altura y es apropiado cuando el runtime proporciona origen de suelo.
- Creamos un GameObject vacío independiente llamado **PuntoInicioVR**, colocado sobre el suelo en el destino elegido.
- Añadimos el script **InicioVR.cs** sobre XR Origin. El ejemplo esperó un frame y utilizó **XROrigin.MoveCameraToWorldLocation()** para alinear horizontalmente la cámara con el punto.
- La prueba no fijó de forma fiable la aparición al conectar el visor real; por eso **no se incluye como solución final garantizada**.

Qué conviene revisar en la próxima sesión

Hipótesis	Comprobación
Tracking aún se estabiliza después de Start	Comprobar Camera pose y tracking state durante varios frames / evento de tracking.
Se ajusta posición pero no orientación	Aplicar la orientación inicial usando API de XR Origin de forma coordinada con el tracking.
Recentrado posterior en Quest Link	Comprobar si el recenter cambia la referencia espacial tras InicioVR.
Simulador y Quest usan referencias distintas	Validar el punto de inicio en ambos modos, sin mover la Main Camera hija manualmente.

ESTADO HONESTO

Este punto **no quedó cerrado** en la sesión. El PDF registra lo que se probó y cómo seguir sin arriesgar el resto del proyecto que ya funciona.

16 · Diagnóstico rápido

Problemas reales que aparecieron y cómo distinguirlos.

Síntoma	Causa a comprobar	Qué hacer
No entra en Play / CS0101, CS0111	Clases repetidas en Assets	Apartar duplicados fuera de Assets; no desactivar objetos solamente.
Rayo blanco / no arco	Está activo Near-Far Interactor, no Teleport Mode	Activar XRI Right Locomotion/Teleport Mode.
Arco sin destino válido	Physics Layer, XR Layer o collider no coinciden	Revisar SueloTeleport, Teleport y Teleportation Area.
Apunta pero no traslada	No está disponible Teleportation Provider	Comprobar Provider + Locomotion Mediator.
No aparecen mandos reales	Perfil OpenXR vacío / seguimiento	Añadir Oculus Touch; revisar Input Action Manager y controlador activo.
Pantalla con solo cielo	Cámara real desplazada/orientada	Revisar tracking, recenter y spawn sin cambiar el escenario.
Mensaje Mono.Cecil	Problema editor/assemblies independiente	Inspeccionar stack trace completo; no confundirlo con un fallo del rayo.
“Falta InputComparativaMovimiento...”	Script viejo de comparativa sigue activo	Quitar/desactivar ese ejercicio de la escena VR; no es parte del XR Origin.

Rutina recomendada de comprobación

1) Salir de Play. **2)** Guardar escena. **3)** Revisar errores críticos. **4)** Elegir modo NASCOR > VR. **5)** Validar solo una funcionalidad por vez: imagen, tracking, manos, locomoción continua, teletransporte.

17 · Reutilizarlo en DeadEarth y otro VR

La arquitectura XR ya está preparada para reutilizarla en proyectos con escenarios reales.

Receta mínima que repetiremos en cada proyecto

Paso	NASCOR	DeadEarth / proyecto VR
1. Preparar	Compila y tiene muestras XRI	Conservar escenas y modelos (incluidos zombis); instalar los mismos paquetes.
2. XR Origin	Un solo prefab XR Origin	Añadir o adaptar uno; desactivar cámara antigua.
3. Entrada	XRI Default Input Actions	Mismo asset; revisar bindings/profiles según visor.
4. Simular	Un XR Device Simulator	Un único prefab y selector de modo.
5. Destinos	Plane + Collider + Teleportation Area	Asignar superficies de suelo concretas y capas.
6. Gafas	OpenXR + Oculus Touch + Link	Misma configuración PC VR y pruebas en el dispositivo.

Qué no hay que copiar ciegamente

- La escena, los colliders, zonas seguras y límites no tienen por qué coincidir entre NASCOR y DeadEarth.
- En escenarios de zombis, el raycast que ignora objetos puede permitir cruzar paredes; valorar oclusión, navmesh/zonas válidas y seguridad antes de usarlo como mecánica final.
- Los materiales de manos importadas pueden verse magenta si el shader no coincide con el render pipeline; revisar compatibilidad URP.
- Comprobar el Animator y el input de cada proyecto: las manos riggeadas no se animan solas por incluir un Skinned Mesh Renderer.

META FINAL

Mismo proyecto con dos experiencias: **Play de Editor usando simulador** para iterar rápido y **Play por Quest Link** para verificar inmersión y comportamiento real.

18 · Checklist final y referencias

Resumen para repetir la práctica sin volver a perder tiempo.

Checklist del proyecto NASCOR

[OK] Starter Assets + XR Device Simulator importados.

[OK] Un XR Origin con camera, controladores, interactores y locomotion.

[OK] Main Camera clásica desactivada; XR Camera del prefab activa.

[OK] Manos del curso como hijos de sus controladores; alternador V / botón A.

[OK] Plane con SueloTeleport, Mesh Collider y Teleportation Area.

[OK] Raycast Mask y XR Interaction Layer configuradas para teletransporte.

[OK] Movimiento continuo + teletransporte en XR Device Simulator.

[OK] Meta Horizon Link: runtime OpenXR activo, escena NASCOR visible en gafas.

[OK] Oculus Touch Controller Profile añadido tras detectar que faltaba.

[PENDIENTE] Inicio espacial exacto al ponerse las Quest.

[PENDIENTE] Comprobar seguimiento final/visuales de ambos mandos reales.

[PENDIENTE] Revisar errores de comparativa/Mono.Cecil y aviso del driver NVIDIA.

Documentación oficial consultada

Unity - Project setup (XRI 3.1): docs.unity.cn/Packages/com.unity.xr.interaction.toolkit@3.1/manual/general-setup.html

Unity - Starter Assets: docs.unity.cn/Packages/com.unity.xr.interaction.toolkit@3.1/manual/samples-starter-assets.html

Unity - XR Device Simulator: docs.unity.cn/Packages/com.unity.xr.interaction.toolkit@3.1/manual/xr-device-simulator.html

Unity - Create a scene with locomotion:
docs.unity.cn/Packages/com.unity.xr.interaction.toolkit@3.1/manual/create-scene-with-locomotion.html

Unity - Meta Quest Link: docs.unity.cn/Packages/com.unity.xr.meta-openxr@2.1/manual/get-started/link.html

Capturas: proporcionadas durante la sesión de NASCOR. Las configuraciones marcadas como “revisar” dependen del Inspector y de la versión efectiva del paquete; no todas fueron comprobadas como definitivas.